Mathematical Programming Computation manuscript No.
(will be inserted by the editor)

Practical Regularized Quasi-Newton Methods with Inexact Function Values

Hiroki Hamaguchi · **Naoki Marumo** ·
Akiko Takeda

Received: date / Accepted: date

Abstract Many practical optimization problems involve objective function values that are corrupted by bounded, nonvanishing numerical errors. We propose a noise-tolerant regularized quasi-Newton method that exploits a relaxed Armijo line search when observed function values provide reliable progress and switches to an objective-function-free regularization mechanism otherwise. With exact gradients and quasi-Newton matrices satisfying a fixed lower and gradient-scaled upper spectral bound, we prove an order-optimal $\mathscr{O}\left(1/\varepsilon^2\right)$ iteration-complexity bound despite bounded relative-or-absolute function errors. Thus, although the method uses persistently noisy function values whenever they certify progress, it retains the best possible worst-case dependence on the stationarity tolerance available in the noise-free L-smooth nonconvex setting. The exact-gradient qualification is essential to the stated theorem; experiments with perturbed gradients are reported separately as deliberately adverse robustness tests beyond its scope. Experiments on CUTEst include function-value-only noise, joint function-and-gradient noise, and 64-, 32-, and 16-bit arithmetic. They evaluate both robustness and the sensitivity to the supplied function-error bound.

Keywords Regularized Quasi-Newton Method · Numerical Stability · Numerical Error · AdaGrad-Norm · Objective-Function-Free Optimization

Mathematics Subject Classification (2020) 90C53 · 90C06 · 65K05 · 49M15

H. Hamaguchi
Graduate School of Information Science and Technology, The University of Tokyo, Tokyo, Japan
E-mail: hirokihamaguchi0331@gmail.com

N. Marumo
Graduate School of Information Science and Technology, The University of Tokyo, Tokyo, Japan

A. Takeda
Graduate School of Information Science and Technology, The University of Tokyo, Tokyo, Japan
Center for Advanced Intelligence Project, RIKEN, Tokyo, Japan

1 Introduction

We consider the unconstrained optimization problem

$$\underset{x\in\mathbb{R}^n}{\text{minimize}} \quad f(x), \tag{1}$$

where $f\colon \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable nonconvex function, and $x \in \mathbb{R}^n$ is the optimization variable. We focus on settings where objective function evaluations are contaminated by bounded and non-diminishing numerical noise [39]. Specifically, only inexact function values $\overline{f}(x)$ are available for optimization. Such inaccuracies are unavoidable in many realistic scenarios, including finite-precision arithmetic, simulation-based evaluations, and stochastic approximations [5, 30, 38]. When function values are contaminated by numerical noise, standard optimization algorithms may lose their theoretical guarantees, behave erratically, or terminate prematurely with unreliable solutions.

In the absence of noise and with accurate function evaluations, quasi-Newton methods, in particular the Broyden–Fletcher–Goldfarb–Shanno (BFGS) method and its limited-memory variant (L-BFGS), are among the most efficient and widely used algorithms for solving problem (1) [26, 32]. Their success is largely attributed to fast local convergence and scalability. Standard implementations [33, 42] rely on line search procedures that enforce the Wolfe conditions to guarantee sufficient descent and the positive definiteness of the Hessian approximation.

In the presence of numerical noise, these line search conditions may become unreliable. Differences in function values or directional derivatives can be dominated by numerical errors, leading to unstable step sizes, ill-conditioned Hessian approximations, or abnormal termination [34, 38]. From a computational perspective, improving the robustness of quasi-Newton methods in noisy environments is therefore crucial. In particular, it is desirable to design algorithms that (i) retain computational efficiency when function evaluations are accurate, (ii) remain stable when function values are unreliable, and (iii) guarantee convergence to first-order stationary points under mild assumptions.

In this work, we propose a noise-tolerant regularized quasi-Newton method that addresses these challenges. Our approach combines a relaxed Armijo line search, quadratic regularization [23, 41], and an OFFO-inspired strategy [17, 19]. Theorem 1 of Kanzow and Steck [23] assumes a bounded sequence of quasi-Newton matrices and an objective function bounded from below and proves $\liminf_{k\to\infty} \|\nabla f(x_k)\| = 0$, so every fixed gradient tolerance is attained after finitely many iterations. Their result establishes global convergence without an iteration-complexity bound, whereas our analysis gives $\mathscr{O}\left(1/\varepsilon^2\right)$ complexity under bounded function-value errors. This dependence on the stationarity tolerance is worst-case optimal for L-smooth nonconvex optimization even in the noise-free setting [**?**]. The theoretical contribution is therefore that the hybrid method retains this best possible order while continuing to exploit persistently noisy function values whenever they provide a reliable progress certificate. Unlike OFFO algorithms, which deliberately never evaluate the objective function, our hybrid method retains a relaxed line search whenever observed function values certify progress and invokes OFFO-type scaling only otherwise. The noise-tolerant quasi-Newton analysis in [38] yields a stationarity neighborhood whose function-error contribution is proportional to $\sqrt{M\varepsilon_f}$, whereas its gradient-error contribution is proportional to ε_g. Its relaxed Armijo condition is closely related to ours; when $|f|$ is small, our hybrid model gives an error-absorbing scale of approximately $2\varepsilon_f/(1-\varepsilon_f)$, while our adaptive recomputation and the K^0/K^+ switch determine when function information is used. Consequently, up to problem-dependent constants, making the former no larger than the latter requires the comparatively restrictive regime $\varepsilon_f = \mathscr{O}\left(\varepsilon_g^2/M\right)$. The difference is particularly clear when gradients are exact: even with $\varepsilon_g = 0$, the neighborhood in [38, Theorem 3.5] retains a stationarity threshold proportional to $\sqrt{M\varepsilon_f}$, whereas in our result a fixed $0 \le \varepsilon_f < 1$ affects the complexity constant but does not impose a stationarity floor. Thus, under exact gradients, our method can reach an arbitrary prescribed first-order tolerance despite fixed nonvanishing relative-or-absolute function errors because it can fall back on an objective-function-free scale when function comparisons are uninformative. This advantage is conditional rather than uniform: Shi et al. allow bounded gradient errors and stabilize noisy curvature pairs through lengthening, whereas our complexity theorem assumes exact gradients. The two results are therefore complementary, with ours providing the stronger attainable-accuracy guarantee for persistent function-value errors under exact gradients and theirs covering the more general case of simultaneous function and gradient errors under strong convexity.

When a sufficient decrease in observed function value is available, the method uses an unregularized quasi-Newton step; otherwise, it uses regularization governed by an AdaGrad-Norm-type accumulator over the regularized iterations. We prove that the resulting method reaches an ε-first-order stationary point in the order-optimal $\mathscr{O}\left(1/\varepsilon^2\right)$ iterations for smooth nonconvex objectives under exact gradients and bounded relative-or-absolute function errors. This matches the best possible worst-case order for the corresponding noise-free class even though the method still uses the available noisy function values selectively. We state the exact-gradient scope explicitly and separate theory-aligned function-noise experiments from joint function-and-gradient perturbations, the latter serving as deliberately adverse stress tests rather than evidence for the theorem. The CUTEst study also reports problem dimensions, the baseline stopping rules, sensitivity to the supplied function-error estimate, and ablations of the hybrid mechanism.

The remainder of this paper is organized as follows. In section 2, we review relevant background and introduce the problem setting. In section 3, we present the proposed algorithm. In section 4, we establish a global convergence property of the proposed algorithm. In section 5, we discuss practical choices of algorithmic variables and parameters. In section 6, we report numerical results on the CUTEst benchmark collection. Finally, in section 7, we conclude the paper with a summary and future research directions.

Our implementation is publicly available on GitHub[1].

[1] https://github.com/HirokiHamaguchi/qnlab

Algorithm	Utilize Function values	Tolerant to Function noise	Gradient noise allowed	Problem class	Guarantee
`Line` / `SciPy` [33, 42]	✓	–	–	nonconvex	–
`Reg` [23]	✓	–	–	nonconvex	$\liminf_{k\to\infty} \lVert\nabla f(x_k)\rVert = 0$
`NTQN` [38, 45]	✓	✓	✓	Strongly convex	Linear convergence to a noise-dependent neighborhood. The stationarity floor contains an $\mathcal{O}\left(\sqrt{M\varepsilon_f}\right)$ function-error term and an $\mathcal{O}(\varepsilon_g)$ gradient-error term
`OFFO` [17]	–	✓✓	–	nonconvex	$\mathcal{O}\left(1/\varepsilon^2\right)$ first-order complexity without function-value evaluations
`Ours` Algorithm 1	✓	✓✓	(practically)	nonconvex	Order-optimal $\mathcal{O}\left(1/\varepsilon^2\right)$ complexity under exact gradients and fixed bounded relative-or-absolute function errors

Table 1: Comparison of the proposed method with its numerical baselines and first-order objective-function-free optimization (OFFO). A ✓ in the Function Values Used column indicates that the algorithm evaluates function values, while a ✓ in a noise column indicates that the corresponding nonvanishing error is covered by the stated theory; a blank entry means that it is not covered. The noise columns describe theoretical assumptions rather than whether noisy experiments were performed. In particular, the guarantee for NTQN contains a function-error-dependent stationarity floor, whereas our guarantee permits fixed nonvanishing relative-or-absolute function errors without such a floor under exact gradients.

2 Preliminaries and Problem Setting

We begin by reviewing relevant background and formalizing the problem setting.

2.1 Regularized Quasi-Newton Method

A regularized quasi-Newton method is a variant of quasi-Newton methods that does not rely heavily on line-search techniques. This property is important for optimization in noisy evaluation environments. There has been renewed interest in regularized Newton-type methods, including cubic-regularization schemes [31, 35, 43], and we focus on quadratic regularization [9, 23, 25, 37, 40, 41, 48] for simplicity of analysis and implementation.

For $g_k := \nabla f(x_k)$ and an approximate Hessian $B_k \in \mathbb{R}^{n\times n}$, regularized quasi-Newton methods compute the step as

$$x_{k+1} = x_k + d_k, \quad d_k = -(B_k + \mu_k I)^{-1} g_k,$$

with a regularization parameter $\mu_k \geq 0$. Combining regularization with line search is also possible and has been explored [40]. Several strategies for choosing μ_k in nonconvex optimization have been proposed [37, 40, 41], and one of them is given by

$$\mu_k = c_1 \max(0, -\lambda_{\min}(B_k)) + c_2 \|\nabla f(x_k)\|^\delta,$$

where $c_1 > 1$, $c_2 > 0$, $\delta \geq 0$, and $\lambda_{\min}(B_k)$ denotes the smallest eigenvalue of B_k. This choice guarantees that $B_k + \mu_k I$ is positive definite, ensuring that d_k is a descent direction. We adapt such regularization strategies to noisy evaluation environments.

2.2 Objective-Function-Free Optimization

Objective-Function-Free Optimization (OFFO) refers to a class of methods that avoid explicit evaluations of the objective function and rely solely on gradient information [17, 19]. Such methods are particularly attractive when function values are unreliable due to noise. Similar adaptive trust-region approaches are also discussed [16].

Prominent examples of OFFO include adaptive gradient methods such as AdaGrad [11] and AdaGrad-Norm [44]. Their update rules are

$$\text{(AdaGrad)} \quad x_{k+1}^{(i)} = x_k^{(i)} - \frac{1}{\theta\sqrt{\varsigma + \sum_{j=0}^{k} (g_j^{(i)})^2}} g_k^{(i)}, \qquad (i = 1, 2, \ldots, n)$$

$$\text{(AdaGrad-Norm)} \quad x_{k+1} = x_k - \frac{1}{\theta\sqrt{\varsigma + \sum_{j=0}^{k} \|g_j\|^2}} g_k$$

respectively, where $\varsigma > 0$ and $\theta > 0$ are algorithmic parameters, and $x^{(i)}$ denotes the i-th component of a vector x.

2.3 Problem Setting

We consider optimization problems in which exact objective function values are unavailable, following the recent literature on error-aware optimization [2, 6, 7, 18, 36, 39]. When both objective values and gradients are heavily corrupted by noise, reliable optimization cannot be guaranteed. Thus, meaningful optimization requires that at least one source of information be sufficiently accurate. This work focuses on problems arising from finite-precision computations or inexact evaluations, where objective function evaluations are subject to non-negligible errors while gradient information can be computed with relatively high accuracy. Settings dominated by gradient errors, such as stochastic approximation and inexact-gradient methods [3, 38, 45], are beyond the scope of this work.

In the following, we formalize the assumptions on function value and gradient evaluations. We assume that only an error-contaminated function value $\overline{f}(x)$ is accessible, satisfying the hybrid absolute-relative error model [21, 22]:

$$\left| f(x) - \overline{f}(x) \right| \leq \varepsilon_{\mathrm{f}} \max(1, |f(x)|), \tag{2}$$

where $0 \leq \varepsilon_{\mathrm{f}} < 1$ is an error rate. The error is predominantly relative when $|f(x)|$ is large and effectively absolute when $|f(x)|$ is small, capturing the typical behavior of rounding errors in IEEE 754 floating-point arithmetic [21]. The choice of ε_{f} is motivated by the practical heuristic used in numerical stopping rules rather than by treating one rounding operation as the total error. For example, SciPy's L-BFGS-B uses $\texttt{ftol} = \texttt{factr}\,\varepsilon_{\mathrm{mach}}$ with the moderate-accuracy default $\texttt{factr} = 10^7$ [42]. We similarly use a precision-dependent multiple of machine epsilon as an aggregate error allowance and assess sensitivity to this supplied value in section 6.

We distinguish the setting covered by the analysis from the additional stress tests in section 6. The convergence theory makes the following exact-gradient assumption.

Assumption 1 (Exact gradient) *The algorithm has access to a gradient oracle g such that, for every evaluated point x,*

$$g(x) = \nabla f(x). \tag{3}$$

The gradient oracle is independent of the inexact function-value oracle $\overline{f}$; in particular, $\overline{f}$ need not be differentiable. This assumption is not implied by the function-value error model in eq. (2) and is therefore stated independently. With a bounded nonvanishing gradient error, exact first-order stationarity is generally impossible to certify below the noise floor; a result in that setting would instead have to bound convergence to a neighborhood. Accordingly, function-value-only noise is the theory-aligned artificial experiment, while joint function-and-gradient noise is reported as a deliberately adverse robustness test beyond the theorem.

3 Proposed Algorithm

We propose a new algorithm for the problem setting stated in section 2.3. We draw inspiration from the OFFO framework to design robust regularization and scaling strategies. Unlike purely OFFO methods, our algorithm exploits function values when they are numerically reliable and smoothly switches to gradient-based control otherwise. A regularized quasi-Newton direction alone does not specify how to update the regularization scale once comparisons of noisy function values become uninformative, whereas the cumulative gradient scale provides such an update without further objective comparisons. Using that scale at every iteration would instead discard useful function information and may be unnecessarily conservative, which motivates the K^0/K^+ switch. This hybrid behavior is intended to balance stability under unreliable function values with the efficiency of unregularized quasi-Newton steps when function information is useful. The pseudocode of our algorithm is given in Algorithm 1. The minimum of the empty set is defined as $+\infty$ in line 3. Algorithm 2 is an auxiliary line search algorithm called in Algorithm 1. In the following, we explain the three main components of Algorithm 1.

Algorithm 1: Noise Tolerant Regularized Quasi-Newton Method

Input: x_0, $0 < k_{\max}$, $0 \le \varepsilon_{\mathrm{f}} < 1$, $0 < \theta_{\min} \le \theta_{\max}$ and $0 < \varsigma < 1$

1 $k \leftarrow 0$, $g_0 \leftarrow g(x_0)$, $K^0 \leftarrow \emptyset$, $K^+ \leftarrow \emptyset$
2 **for** $k = 0, 1, 2, \dots, k_{\max} - 1$ **do**
3   **if** $\min_{j \in K^0,\, j<k} (\overline{f}(x_j) - \Delta_j) \ge \overline{f}(x_k)$ **then**
4         $K^0 \leftarrow K^0 \cup \{k\}$
5         $\mu_k \leftarrow 0$
6   **else**
7         $K^+ \leftarrow K^+ \cup \{k\}$
8         $\mu_k \leftarrow \theta_k \sqrt{\varsigma + \sum_{j \in K^+,\, j \le k} \|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ (see section 5.3).
9     $d_k \leftarrow -(B_k + \mu_k I)^{-1} g_k$ with B_k (see section 5.1).
10   Find α_k and Δ_k satisfying eq. (4) by Algorithm 2.
11     $x_{k+1} \leftarrow x_k + \alpha_k d_k$, $g_{k+1} \leftarrow g(x_{k+1})$
12 **end**

Algorithm 2: Backtracking Line Search with Relaxed Armijo Condition

Input: $x_k \in \mathbb{R}^n$, $d_k \in \mathbb{R}^n$, $g_k \in \mathbb{R}^n$, $0 < c < 1$ and $0 < \beta_{\min} < \beta_{\max} < 1$

1 $\alpha_k \leftarrow 1$
2 $\Delta_k \leftarrow \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k))$
3 **while** $\overline{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k < \overline{f}(x_k + \alpha_k d_k)$ **do**
4     $\alpha_k \leftarrow \beta \alpha_k$ with $\beta \in [\beta_{\min}, \beta_{\max}]$ (see section 5.2).
5     $\Delta_k \leftarrow \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k))$
6 **end**
7 **return** α_k

Computation of the Regularized Quasi-Newton Direction (line 9): The first component is the computation of the regularized quasi-Newton direction d_k stated in section 2.1. Given $g_k = g(x_k) = \nabla f(x_k)$, an approximate Hessian B_k, and a regularization parameter μ_k, we compute the search direction $d_k = -(B_k + \mu_k I)^{-1} g_k$. A practical choice of B_k is discussed in section 5.1.

Relaxed Armijo Condition with an Error-Absorbing Term (line 10): The second component is the computation of the step size α_k using a relaxed Armijo condition, which is defined as follows:

$$\begin{cases} \overline{f}(x_k) + c\alpha_k g_k^\top d_k + \Delta_k \ge \overline{f}(x_k + \alpha_k d_k), \\ \Delta_k = \dfrac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k + \alpha_k d_k)), \end{cases} \tag{4}$$

where Δ_k is the error-absorbing term that is recomputed for each α_k. This relaxation guarantees the existence of $\alpha_k \in (0, 1]$ even in the presence of errors in $\overline{f}$. When d_k

is a descent direction ($g_k^\top d_k < 0$), eq. (4) ensures that a temporary increase of $\overline{f}$ is at most Δ_k. Since Δ_k depends on $-\overline{f}(x_k + \alpha_k d_k)$ but not on $\overline{f}(x_k + \alpha_k d_k)$, Δ_k does not increase when $\overline{f}(x_k + \alpha_k d_k)$ becomes large. This prevents unbounded growth of Δ_k.

Adaptive Update of the Regularization Parameter (lines 3–8): The third component is the adaptive update of the regularization parameter μ_k. A larger μ_k yields a more conservative step, while a smaller μ_k allows a more aggressive quasi-Newton step, especially when $\mu_k = 0$. In practical settings, $\mu_k = 0$ is preferred, since this allows us to fully exploit the quasi-Newton approximation and typically leads to practically faster convergence. Let us partition the iteration indices $K := \{0, 1, 2, \ldots, k_{\max} - 1\}$ into two sets:

$$K^0 := \{k \in K \mid \mu_k = 0\}, \quad K^+ := \{k \in K \mid \mu_k > 0\}. \tag{5}$$

We set $\mu_k = 0$ if and only if the following condition is satisfied:

$$\min_{j \in K^0,\, j<k} \left(\overline{f}(x_j) - \Delta_j\right) \geq \overline{f}(x_k). \tag{6}$$

Note that $k = 0$ satisfies eq. (6) since K^0 is empty. This condition restricts $\mu_k = 0$ to iterations for which a sufficient decrease in $\overline{f}$ has been observed. Thus, although $\overline{f}$ can temporarily increase, this condition ensures that $\overline{f}$ does not grow without bound or oscillate indefinitely. Otherwise, to ensure convergence to first-order stationary points, we apply an OFFO-based update

$$\mu_k = \theta_k \sqrt{\varsigma + \sum_{j \in K^+,\, j \leq k} \|g_j\|^2}, \tag{7}$$

as described in section 2.2. Since OFFO schemes do not rely on function values, this update is robust to numerical errors in $\overline{f}$.

4 Theoretical Analysis

In this section, we establish the global convergence properties of Algorithm 1 under the problem setting in section 2.3, namely, eqs. (2) and (3).

4.1 Assumptions

In addition to the exact-gradient assumption stated in Assumption 1, we make the following three structural assumptions. The first assumption ensures that the objective function is bounded below.

Assumption 2 *The function f is bounded below, meaning that for all $x \in \mathbb{R}^n$,*

$$f(x) \geq f^* := \inf_{x \in \mathbb{R}^n} f(x) > -\infty.$$

By the triangle inequality and eq. (2), we have $\overline{f}(x) \geq f(x) - \varepsilon_{\mathrm{f}} \max(1, |f(x)|)$. Since the function $g(t) = t - \varepsilon_{\mathrm{f}} \max(1, |t|)$ is non-decreasing for $0 \leq \varepsilon_{\mathrm{f}} < 1$, it follows that $\inf_x g(f(x)) = g(f^*)$. Consequently, we can also lower bound $\overline{f}$ as

$$\overline{f}(x) \geq \overline{f}^* := \inf_{x \in \mathbb{R}^n} (f(x) - \varepsilon_{\mathrm{f}} \max(1, |f(x)|)) = f^* - \varepsilon_{\mathrm{f}} \max(1, |f^*|) > -\infty. \tag{8}$$

The second assumption ensures smoothness of the objective function.

Assumption 3 *The function $f\colon \mathbb{R}^n \to \mathbb{R}$ is L-smooth, meaning that there exists a constant $L > 0$ such that for all $x, y \in \mathbb{R}^n$,*

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2.$$

Set $\omega_{\min} := \min\{1, \sqrt{\varsigma}\} > 0$ and $\omega_k := \max\{\omega_{\min}, \|g_k\|\}$. The third assumption requires the matrices $\{B_k\}_{k=0}^{k_{\max}-1}$ to be uniformly positive definite and upper bounded relative to ω_k.

Assumption 4 *There exist constants $0 < m \leq M$ such that all B_k satisfy*

$$mI \preceq B_k \preceq M\omega_k I.$$

As we will discuss in section 5, the standard BFGS update with a slight modification guarantees Assumption 4 in practice. By Assumption 4, we can derive useful bounds involving g_k and d_k. Using $d_k = -(B_k + \mu_k I)^{-1} g_k$ (line 9) and Assumption 4 asserting $(m + \mu_k)I \preceq B_k + \mu_k I \preceq (M\omega_k + \mu_k)I$, we obtain the following bounds:

$$-g_k^\top d_k = d_k^\top (B_k + \mu_k I) d_k \geq (m + \mu_k)\|d_k\|^2 \geq m\|d_k\|^2, \tag{9}$$

$$-g_k^\top d_k = g_k^\top (B_k + \mu_k I)^{-1} g_k \geq \frac{1}{M\omega_k + \mu_k}\|g_k\|^2, \tag{10}$$

$$\|d_k\|^2 = \left\|(B_k + \mu_k I)^{-1} g_k\right\|^2 \leq \frac{1}{(m + \mu_k)^2}\|g_k\|^2. \tag{11}$$

4.2 Error Bound on Function Evaluations

We first establish useful properties of the error-absorbing term Δ_k defined in eq. (4). In the following, we define the actual and observed function decreases as

$$\delta_k := f(x_k) - f(x_{k+1}), \tag{12}$$

$$\overline{\delta}_k := \overline{f}(x_k) - \overline{f}(x_{k+1}). \tag{13}$$

The first property states that the difference between the actual decrease δ_k and the observed decrease $\overline{\delta}_k$ can be bounded by Δ_k.

Lemma 1 *If x_k and x_{k+1} satisfy $f(x_{k+1}) \leq f(x_k)$, it holds that*

$$\delta_k \leq \overline{\delta}_k + \Delta_k.$$

Proof We start from general observations. We first prove that, for all $x \in \mathbb{R}^n$,

$$\begin{cases} \max(1, +f(x)) \leq \frac{1}{1-\varepsilon_\mathrm{f}} \max(1, +\overline{f}(x)), \\ \max(1, -f(x)) \leq \frac{1}{1-\varepsilon_\mathrm{f}} \max(1, -\overline{f}(x)). \end{cases} \tag{14}$$

We prove the inequality with the plus sign. If $+f(x) \leq 1$, then $\max(1, +f(x)) = 1 \leq \frac{1}{1-\varepsilon_\mathrm{f}} \leq \frac{1}{1-\varepsilon_\mathrm{f}} \max(1, +\overline{f}(x))$. If $+f(x) > 1$, eq. (2) implies

$$\left|f(x) - \overline{f}(x)\right| \leq \varepsilon_\mathrm{f} \max(1, |f(x)|) = +\varepsilon_\mathrm{f} f(x) \tag{15}$$

and thus $f(x) - \overline{f}(x) \leq +\varepsilon_\mathrm{f} f(x)$. Hence,

$$\max(1, +f(x)) = +f(x) \leq +\frac{1}{1-\varepsilon_\mathrm{f}} \overline{f}(x) \leq \frac{1}{1-\varepsilon_\mathrm{f}} \max(1, +\overline{f}(x)).$$

The case with the negative sign can be shown analogously, using $-f(x) + \overline{f}(x) \leq -\varepsilon_\mathrm{f} f(x)$, which follows from eq. (15). This completes the proof of eq. (14).

We also have the following general bound for all $k \in K$:

$$\begin{aligned} \left|\delta_k - \overline{\delta}_k\right| &= \left|(f(x_k) - \overline{f}(x_k)) - (f(x_{k+1}) - \overline{f}(x_{k+1}))\right| && \text{(rearrangement)} \\ &\leq \left|f(x_k) - \overline{f}(x_k)\right| + \left|f(x_{k+1}) - \overline{f}(x_{k+1})\right| && \text{(triangle inequality)} \\ &\leq \varepsilon_\mathrm{f} \max(1, |f(x_k)|) + \varepsilon_\mathrm{f} \max(1, |f(x_{k+1})|) && \text{(eq. (2))} \\ &\leq 2\varepsilon_\mathrm{f} \max(1, |f(x_k)|, |f(x_{k+1})|). && \text{(max property)} \end{aligned} \tag{16}$$

With these preparations, we now prove the lemma. From the condition $f(x_{k+1}) \leq f(x_k)$, eq. (16) can be further bounded as

$$\begin{aligned} 2\varepsilon_\mathrm{f} \max(1, |f(x_k)|, |f(x_{k+1})|) &= 2\varepsilon_\mathrm{f} \max(1, f(x_k), -f(x_{k+1})) && (f(x_{k+1}) \leq f(x_k)) \\ &\leq \frac{2\varepsilon_\mathrm{f}}{1-\varepsilon_\mathrm{f}} \max(1, \overline{f}(x_k), -\overline{f}(x_{k+1})) && \text{(eq. (14))} \\ &= \Delta_k, && \text{(eq. (4))} \end{aligned}$$

which yields the desired inequality with $\delta_k - \overline{\delta}_k \leq \left|\delta_k - \overline{\delta}_k\right| \leq \Delta_k$. □

We also derive the following lemma for later use.

Lemma 2 *Under Assumption 4, it holds that*

$$\overline{\delta}_k \leq \delta_k + \frac{1+\varepsilon_\mathrm{f}}{1-\varepsilon_\mathrm{f}} \Delta_k.$$

Proof Since $-g_k^\top d_k \geq 0$ from Assumption 4 and eq. (9), the relaxed Armijo condition (eq. (4)) implies

$$\overline{f}(x_k) - \overline{f}(x_{k+1}) \geq -c\alpha_k g_k^\top d_k - \Delta_k \geq -\Delta_k. \tag{17}$$

Thus, using the general results in eq. (16) and $\Delta_k \geq 0$, we can further bound as

$$\left|\delta_k - \overline{\delta}_k\right| \leq 2\varepsilon_\mathrm{f} \max(1, |f(x_k)|, |f(x_{k+1})|) \qquad \text{(eq. (16))}$$

$$
\begin{aligned}
&\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k), -\overline{f}(x_k), \overline{f}(x_{k+1}), -\overline{f}(x_{k+1})) && \text{(eq. (14))} \\
&\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \max(1, \overline{f}(x_k) + \Delta_k, -\overline{f}(x_{k+1}) + \Delta_k) && \text{(eq. (17))} \\
&\le \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}} \left(\max(1, \overline{f}(x_k), -\overline{f}(x_{k+1})) + \Delta_k\right) \\
&= \left(1 + \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\right)\Delta_k = \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k, && \text{(eq. (4))}
\end{aligned}
$$

which yields the desired inequality with $\overline{\delta}_k - \delta_k \le \left|\delta_k - \overline{\delta}_k\right| \le \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k$. □

4.3 Backtracking Stage

We next analyze the line search in Algorithm 2. Recall that the parameter c in Algorithm 2 satisfies $0 < c < 1$ and $0 < m$ from Assumption 4.

Lemma 3 *Under Assumptions 3 and 4, the backtracking procedure in Algorithm 2 terminates whenever the step size α_k satisfies*

$$\alpha_k \le \frac{2(1-c)m}{L}. \tag{18}$$

Proof Under the L-smoothness of f (Assumption 3), we always have

$$
\begin{aligned}
f(x_k) - f(x_k + \alpha_k d_k) &\ge -\alpha_k g_k^\top d_k - \frac{L}{2}\alpha_k^2 \|d_k\|^2 \\
&= -c\alpha_k g_k^\top d_k + \alpha_k \left(-(1-c) g_k^\top d_k - \frac{L\alpha_k}{2}\|d_k\|^2\right).
\end{aligned} \tag{19}
$$

From Assumption 4 and eq. (9), we have $-g_k^\top d_k \ge m\|d_k\|^2$. Under eq. (18), we have

$$-(1-c)g_k^\top d_k - \frac{L\alpha_k}{2}\|d_k\|^2 \ge \left((1-c)m - \frac{L\alpha_k}{2}\right)\|d_k\|^2 \ge 0. \tag{20}$$

Combining eqs. (19) and (20) yields

$$f(x_k) - f(x_k + \alpha_k d_k) \ge -c\alpha_k g_k^\top d_k.$$

Since $-g_k^\top d_k \ge 0$ from Assumption 4 and eq. (9), the condition of Lemma 1 is satisfied with $x_{k+1} = x_k + \alpha_k d_k$. Thus, we obtain $\delta_k \le \overline{\delta}_k + \Delta_k$, i.e.,

$$-c\alpha_k g_k^\top d_k \le f(x_k) - f(x_k + \alpha_k d_k) \le \overline{f}(x_k) - \overline{f}(x_k + \alpha_k d_k) + \Delta_k.$$

This inequality means that the relaxed Armijo condition (eq. (4)) is satisfied and thus the backtracking procedure terminates. This completes the proof. □

Now, let us analyze Algorithm 2 using Lemma 3. Define

$$\overline{\alpha} := \min\left(\beta_{\min}\frac{2(1-c)m}{L}, 1\right) > 0.$$

The following states the key property of the backtracking procedure.

Proposition 1 *Under Assumptions 3 and 4, the backtracking line search in Algorithm 2 terminates after finitely many rejections, and the step size α_k satisfies*

$$\alpha_k \geq \overline{\alpha}.$$

Proof Line 4 of Algorithm 2 multiplies α_k by β at each rejection, where $0 < \beta_{\min} \leq \beta \leq \beta_{\max} < 1$. For the number of backtracking rejections r, we have $\alpha_k \leq \beta_{\max}^r$. Thus, r is finite since the backtracking procedure terminates for sufficiently small α_k by Lemma 3. If $r = 0$, $\alpha_k = 1 \geq \overline{\alpha}$. Otherwise, $\alpha_k \geq \beta_{\min}\alpha_k'$, where α_k' is the last rejected step size. By Lemma 3, we have $\alpha_k' > \frac{2(1-c)m}{L}$, and thus

$$\alpha_k \geq \beta_{\min}\alpha_k' > \beta_{\min}\frac{2(1-c)m}{L} \geq \overline{\alpha}.$$

This completes the proof. □

4.4 Bound for the case $\mu_k = 0$

In this subsection, we lower bound the sum of actual function decreases δ_k over $k \in K^0$. Recall that K^0 denotes the set of iteration indices k such that $\mu_k = 0$, as in eq. (5). To this end, we establish that the sum of Δ_k over $k \in K^0$ is upper bounded, where Δ_k is the error-absorbing term in the relaxed Armijo condition (eq. (4)). We introduce the following constant $\Delta_{\max}$, which serves as an upper bound for Δ_k with $k \in K^0$.

$$\Delta_{\max} := \frac{2\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\max(1, \overline{f}(x_0), -\overline{f}^*).$$

Lemma 4 *Under Assumptions 2 to 4, the sum of Δ_k over $k \in K^0$ satisfies*

$$\sum_{k\in K^0} \Delta_k \leq \overline{f}(x_0) - \overline{f}^* + \Delta_{\max}.$$

Proof Let $(k_i)_{i=0}^{\ell}$ with $\ell \geq 0$ be the elements of K^0 in increasing order. We have $\overline{f}(x_{k_\ell}) \leq \overline{f}(x_0)$ from eq. (6) with $k = k_\ell$, where the case $\ell = 0$ follows from $k_0 = 0$. Combining this with Assumption 2, we have $\Delta_{k_\ell} \leq \Delta_{\max}$. For all $i < \ell$, eq. (6) with $j = k_i$ and $k = k_{i+1}$ yields

$$\Delta_{k_i} \leq \overline{f}(x_{k_i}) - \overline{f}(x_{k_{i+1}}), \tag{21}$$

and summing Δ_k over $k \in K^0$ gives

$$\sum_{k\in K^0} \Delta_k \leq \sum_{i=0}^{\ell-1}(\overline{f}(x_{k_i}) - \overline{f}(x_{k_{i+1}})) + \Delta_{k_\ell} \qquad \text{(eq. (21))}$$

$$
\begin{aligned}
&= \overline{f}(x_0) - \overline{f}(x_{k_\ell}) + \Delta_{k_\ell} && \text{(telescoping sum)} \\
&\leq \overline{f}(x_0) - \overline{f}^* + \Delta_{\max}. && (-\overline{f}(x_{k_\ell}) \leq -\overline{f}^* \text{ and } \Delta_{k_\ell} \leq \Delta_{\max})
\end{aligned}
$$

This completes the proof. □

We now bound the sum of $\delta_k = f(x_k) - f(x_{k+1})$ over $k \in K^0$ with $\|g_k\|^2/\omega_k$ as follows.

Lemma 5 *Under Assumptions 2 to 4, the sum of δ_k over $k \in K^0$ satisfies*

$$
\sum_{k\in K^0} \delta_k \geq \frac{c\overline{\alpha}}{M} \sum_{k\in K^0} \frac{\|g_k\|^2}{\omega_k} - \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right).
$$

Proof The relaxed Armijo condition (eq. (4)) at iteration $k \in K^0$ yields

$$
\begin{aligned}
\overline{\delta}_k + \Delta_k &= \overline{f}(x_k) - \overline{f}(x_{k+1}) + \Delta_k && \text{(eq. (13))} \\
&\geq -c\alpha_k g_k^\top d_k && \text{(eq. (4))} \\
&\geq \frac{c\overline{\alpha}}{M\omega_k + \mu_k}\|g_k\|^2 && \text{(Proposition 1 and eq. (10))} \\
&= \frac{c\overline{\alpha}}{M\omega_k}\|g_k\|^2. && (\mu_k = 0 \text{ since } k \in K^0) \qquad (22)
\end{aligned}
$$

Using Lemma 2, we also have

$$
\overline{\delta}_k + \Delta_k \leq \delta_k + \frac{1+\varepsilon_{\mathrm{f}}}{1-\varepsilon_{\mathrm{f}}}\Delta_k + \Delta_k = \delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\Delta_k. \tag{23}
$$

Then, combining eqs. (22) and (23) and summing over $k \in K^0$ gives

$$
\sum_{k\in K^0} \frac{c\overline{\alpha}}{M\omega_k}\|g_k\|^2 \leq \sum_{k\in K^0} \delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\sum_{k\in K^0} \Delta_k \leq \sum_{k\in K^0} \delta_k + \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right),
$$

where the last inequality follows from Lemma 4. Rearranging this completes the proof. □

4.5 Bound for the case $\mu_k > 0$

Next, we show that the sum of δ_k over $k \in K^+$ is lower bounded. Recall that K^+ is the set of iteration indices k such that $\mu_k > 0$ as defined in eq. (5), and that μ_k is defined as $\theta_k\sqrt{\varsigma + \sum_{j\in K^+,\, j\leq k}\|g_j\|^2}$ with $\theta_k \in [\theta_{\min}, \theta_{\max}]$ in line 8 of Algorithm 1.

Before proceeding to the main proof, we present Lemma 6, providing standard inequalities in the AdaGrad-Norm algorithm analysis [44, Lemma 3.2], [17, Lemma 3.1]. Its proof is deferred to appendix A.

Lemma 6 *In Algorithm 1, we have*

$$\sum_{k\in K^+}\frac{\|g_k\|^2}{\mu_k^2}\le\frac{1}{\theta_{\min}^2}\left(\ln\left(\varsigma+\sum_{k\in K^+}\|g_k\|^2\right)-\ln(\varsigma)\right),$$
$$\sum_{k\in K^+}\frac{\|g_k\|^2}{\mu_k}\ge\frac{1}{\theta_{\max}}\left(\sqrt{\varsigma+\sum_{k\in K^+}\|g_k\|^2}-\sqrt{\varsigma}\right).$$

We now lower bound the sum of δ_k over $k\in K^+$ with $\|g_k\|^2$. We define the following constants for later use:

$$D:=1+\frac{M}{\theta_{\min}},\qquad C_1:=\frac{\overline{\alpha}}{D\theta_{\max}},\qquad C_2:=\frac{L}{2\theta_{\min}^2}.$$

Lemma 7 *Under Assumptions 3 and 4, the sum of δ_k over $k\in K^+$ satisfies*

$$\sum_{k\in K^+}\delta_k\ge C_1\left(\sqrt{\varsigma+\sum_{k\in K^+}\|g_k\|^2}-\sqrt{\varsigma}\right)-C_2\left(\ln\left(\varsigma+\sum_{k\in K^+}\|g_k\|^2\right)-\ln(\varsigma)\right).$$

Proof From the L-smoothness (Assumption 3), for $k\in K^+$ we have

$$\begin{aligned}
\delta_k&=f(x_k)-f(x_k+\alpha_k d_k) && (\text{eq. (12) and } x_{k+1}=x_k+\alpha_k d_k)\\
&\ge-\alpha_k g_k^\top d_k-\frac{\alpha_k^2 L}{2}\|d_k\|^2 && (\text{Assumption 3})\\
&\ge\left(\frac{\alpha_k}{M\omega_k+\mu_k}-\frac{\alpha_k^2 L}{2(m+\mu_k)^2}\right)\|g_k\|^2 && (\text{eqs. (10) and (11)})\\
&\ge\left(\frac{\overline{\alpha}}{M\omega_k+\mu_k}-\frac{L}{2(m+\mu_k)^2}\right)\|g_k\|^2. && (\text{Proposition 1 and } \alpha_k\le 1)
\end{aligned}\tag{24}$$

Since $\omega_{\min}\le\sqrt{\varsigma}$ and the accumulator defining μ_k includes the current gradient, we have $\omega_k\le\mu_k/\theta_{\min}$ for $k\in K^+$. We can therefore bound the terms in eq. (24) as

$$\frac{\overline{\alpha}}{M\omega_k+\mu_k}\ge\frac{\overline{\alpha}}{D\mu_k},\quad\frac{L}{2(m+\mu_k)^2}\le\frac{L}{2\mu_k^2}.\tag{25}$$

Thus, applying eq. (25) to eq. (24) yields

$$\delta_k\ge\left(\frac{\overline{\alpha}}{D\mu_k}-\frac{L}{2\mu_k^2}\right)\|g_k\|^2.\tag{26}$$

Summing eq. (26) over $k\in K^+$ and applying Lemma 6 yield

$$\begin{aligned}
\sum_{k\in K^+}\delta_k&\ge\sum_{k\in K^+}\left(\frac{\overline{\alpha}}{D\mu_k}-\frac{L}{2\mu_k^2}\right)\|g_k\|^2\\
&\ge\frac{\overline{\alpha}}{D\theta_{\max}}\left(\sqrt{\varsigma+\sum_{k\in K^+}\|g_k\|^2}-\sqrt{\varsigma}\right)-\frac{L}{2\theta_{\min}^2}\left(\ln\left(\varsigma+\sum_{k\in K^+}\|g_k\|^2\right)-\ln(\varsigma)\right),
\end{aligned}$$

which yields the desired result. □

4.6 Global Convergence Rate

Finally, we combine the results for $k \in K^0$ stated in Lemma 5 and $k \in K^+$ stated in Lemma 7 to derive an upper bound on the total sum of $\|g_k\|^2$. We define a constant F as follows:

$$F := f(x_0) - f^* + \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right) + C_1\sqrt{\varsigma} - C_2\ln(\varsigma). \tag{27}$$

Then, we can obtain the following unified bound for the two types of iterations.

Lemma 8 *Under Assumptions 2 to 4, the sum of $\|g_k\|^2$ over $k \in K = K^0 \cup K^+$ satisfies*

$$F \geq \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\frac{\|g_k\|^2}{\omega_k} + C_1\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - C_2\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right),$$

Proof Since $f(x_{k_{\max}}) \geq f^*$ by Assumption 2 and $K = K^0 \cup K^+$, we have

$$f(x_0) - f^* \geq f(x_0) - f(x_{k_{\max}}) = \sum_{k\in K}\delta_k = \sum_{k\in K^0}\delta_k + \sum_{k\in K^+}\delta_k.$$

From Lemmas 5 and 7, we can further bound as

$$\begin{aligned} f(x_0) - f^* \geq{}& \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\frac{\|g_k\|^2}{\omega_k} - \frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0) - \overline{f}^* + \Delta_{\max}\right) \\ &+ C_1\sqrt{\varsigma + \sum_{k\in K^+}\|g_k\|^2} - C_1\sqrt{\varsigma} - C_2\ln\left(\varsigma + \sum_{k\in K^+}\|g_k\|^2\right) + C_2\ln(\varsigma). \end{aligned}$$

Substituting eq. (27) yields the desired bound, concluding the proof. □

Now, we establish the following iteration-complexity result for Algorithm 1.

Theorem 1 *Under Assumptions 1 to 4, for every $0 < \varepsilon \leq 1$, Algorithm 1 generates an iterate satisfying $\|g_k\| \leq \varepsilon$ within $\mathscr{O}\left(1/\varepsilon^2\right)$ iterations.*

Proof Let us define the function $\phi : \mathbb{R}_{>0} \to \mathbb{R}$ as

$$\phi(G) := \frac{C_1}{2}\sqrt{G} - C_2\ln(G). \tag{28}$$

Since $\phi'(G) = \frac{C_1}{4\sqrt{G}} - \frac{C_2}{G}$, its minimum over $G > 0$ is attained at $G^* := \left(\frac{4C_2}{C_1}\right)^2$ with

$$\phi(G^*) = 2C_2\left(1 - \ln\left(\frac{4C_2}{C_1}\right)\right) = 2C_2\ln\left(\frac{\mathrm{e}C_1}{4C_2}\right),$$

where e is the base of the natural logarithm. Thus, Lemma 8 yields

$$F \geq \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\frac{\|g_k\|^2}{\omega_k} + \frac{C_1}{2}\sqrt{\sum_{k\in K^+}\|g_k\|^2} + \phi\left(\sum_{k\in K^+}\|g_k\|^2 + \varsigma\right)$$

$$\geq \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\frac{\|g_k\|^2}{\omega_k}+\frac{C_1}{2}\sqrt{\sum_{k\in K^+}\|g_k\|^2}+\phi(G^*),$$

which is equivalent to

$$0 \leq \frac{c\overline{\alpha}}{M}\sum_{k\in K^0}\frac{\|g_k\|^2}{\omega_k}+\frac{C_1}{2}\sqrt{\sum_{k\in K^+}\|g_k\|^2} \leq F' \tag{29}$$

where

$$\begin{aligned} F' &:= F-\phi(G^*) \\ &= f(x_0)-f^*+\frac{2}{1-\varepsilon_{\mathrm{f}}}\left(\overline{f}(x_0)-\overline{f}^*+\Delta_{\max}\right)+C_1\sqrt{\varsigma}-C_2\ln(\varsigma)-2C_2\ln\left(\frac{\mathrm{e}C_1}{4C_2}\right). \end{aligned}$$

Suppose that $\|g_k\| > \varepsilon$ for every $0 \leq k < k_{\max}$, where $0 < \varepsilon \leq 1$. Since $\omega_{\min} \leq 1$, the definition of ω_k gives $\|g_k\|^2/\omega_k > \varepsilon^2$ for $k \in K^0$. We also have $\sqrt{\sum_{k\in K^+}\|g_k\|^2} > \varepsilon\sqrt{|K^+|}$. Applying these two bounds to eq. (29) yields

$$\left|K^0\right| < \frac{MF'}{c\overline{\alpha}\varepsilon^2}, \qquad \left|K^+\right| < \frac{4(F')^2}{C_1^2\varepsilon^2}.$$

Since $k_{\max} = \left|K^0\right| + |K^+|$, there must therefore be an iterate with $\|g_k\| \leq \varepsilon$ whenever

$$k_{\max} \geq \frac{1}{\varepsilon^2}\left(\frac{MF'}{c\overline{\alpha}}+\frac{4(F')^2}{C_1^2}\right). \tag{30}$$

This completes the proof. □

As a remark, the function $\phi(G)$ in eq. (28) is closely related to the Lambert W function [8, 17]. For $x \in [-1/\mathrm{e}, 0)$, the equation $ye^y = x$ admits two real negative solutions, and the smaller one is given by the second branch of the Lambert W function, $y = W_{-1}(x) < 0$. Using this function, the equation $\phi(G) = 0$ can be solved in closed form, which leads to a sharper bound in Theorem 1. Since this is not essential for the main argument, we omit the details and refer to [17].

We next interpret eq. (30) in the proof of Theorem 1. The contribution of the indices in K^0 uses the scale-adjusted quantity $\|g_k\|^2/\omega_k$, which equals $\|g_k\|^2/\omega_{\min}$ near stationarity and therefore retains the classical first-order stationarity order. Indeed, for gradient descent with fixed step size $\alpha = 1/L$, one has

$$\frac{1}{k_{\max}}\sum_{k=0}^{k_{\max}-1}\|g_k\|^2 \leq \frac{1}{k_{\max}}\frac{2(f(x_0)-f^*)}{\alpha}.$$

Thus, the term associated with K^0 reflects the standard $\mathscr{O}(f(x_0)-f^*)$ dependence of first-order methods since F' contains the initial gaps $f(x_0)-f^*$ and $\overline{f}(x_0)-\overline{f}^*$. In contrast, the contribution of K^+ resembles the behavior of AdaGrad-Norm, whose convergence bound scales quadratically with the initial optimality gap [44].

Finally, Theorem 1 implies the iteration-complexity bound

$$\min\left\{k \mid \|\nabla f(x_k)\| \le \varepsilon_{\text{gtol}}\right\} = \mathscr{O}\left(1/\varepsilon_{\text{gtol}}^2\right).$$

The order $\mathscr{O}\left(1/\varepsilon_{\text{gtol}}^2\right)$ is worst-case optimal for first-order stationarity on the class of L-smooth nonconvex functions even when function values are exact [**?**]. Hence, bounded nonvanishing errors in the function values used by our hybrid mechanism do not worsen the best possible dependence on $\varepsilon_{\text{gtol}}$, although they affect the constants below. More explicitly, eq. (30) gives the sufficient bound

$$k_{\max} \ge \frac{1}{\varepsilon_{\text{gtol}}^2}\left(\frac{MF'}{c\overline{\alpha}} + \frac{4(F')^2}{C_1^2}\right).$$

Here $\overline{\alpha}$ depends only on c, L, m and the backtracking bounds, $D = 1 + M/\theta_{\min}$, $C_1 = \overline{\alpha}/(D\theta_{\max})$, $C_2 = L/(2\theta_{\min}^2)$, and F' is displayed in the proof and contains ε_{f}, ς, the initial true and observed optimality gaps, and $\Delta_{\max}$. The construction in section 5.1 gives $m = \omega_{\min}$ and $M = 1 + (1+p)\Lambda$, so the memory length enters the sufficient iteration bound through M. Thus the order symbol suppresses no dependence on the iteration counter, but it does suppress these problem and algorithm parameters; the guarantee deteriorates as the permitted function error approaches 1 through the factor $(1-\varepsilon_{\text{f}})^{-1}$.

5 Practical Choices of Algorithmic Variables

In sections 3 and 4, we introduced Algorithm 1 and established its convergence properties. In this section, we discuss practical choices of algorithmic variables and parameters that lead to fast and stable performance. The quantities to be discussed here are the matrices B_k, the step sizes α_k, and the regularization parameters μ_k.

5.1 Approximate Hessian B_k

We first discuss the construction of the matrices B_k used in line 9 of Algorithm 1. The purpose of this subsection is to show that a scaled L-BFGS construction with modified and carefully selected curvature pairs can satisfy Assumption 4.

Construction of B_k: We recall the L-BFGS construction [4, 26, 32]. For an iteration k, we define

$$s_k := x_{k+1} - x_k, \qquad y_k := g_{k+1} - g_k$$

where g_k and g_{k+1} are the gradients at x_k and x_{k+1}, respectively. Starting from an initial matrix, an approximate Hessian is obtained by successively applying BFGS updates to a sequence of curvature pairs. Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a collection of curvature pairs, where k_i denotes the iteration index at which the i-th pair was generated. Starting from the initial matrix $B^{(0)} = \gamma I$ with $\gamma = \left\|y_{k_0}\right\|^2 / y_{k_0}^\top s_{k_0}$, we define

$\mathrm{BFGS}(\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1})$ as the matrix $B^{(p)}$ obtained by sequentially applying the BFGS update as

$$B^{(j+1)} = B^{(j)} - \frac{B^{(j)} s_{k_j} s_{k_j}^\top B^{(j)}}{s_{k_j}^\top B^{(j)} s_{k_j}} + \frac{y_{k_j} y_{k_j}^\top}{y_{k_j}^\top s_{k_j}}$$

for $j = 0, 1, \ldots, p-1$, where each curvature pair (s_{k_j}, y_{k_j}) is applied in order. Here, $p > 0$ denotes the maximum number of stored curvature pairs and is a fixed small integer in the L-BFGS method. When fewer than p curvature pairs are available, the BFGS update is applied using all currently available pairs.

We now provide a sufficient condition for Assumption 4 in the following proposition, which is a variant of existing results [13, Theorem 5.5] and [15, Lemma 1]. Its proof is deferred to appendix B.

Proposition 2 *Let $\{(s_{k_i}, y_{k_i})\}_{i=0}^{p-1}$ be a fixed number of curvature pairs with $s_{k_i} \neq 0$. Assume that there exists a constant $\Lambda > 0$ such that every pair satisfies*

$$y^\top s \geq \frac{1}{\Lambda} \|y\|^2 > 0. \tag{31}$$

Then

$$0 \prec \mathrm{BFGS}(\{s_{k_i}, y_{k_i}\}_{i=0}^{p-1}) \preceq (1+p)\Lambda I. \tag{32}$$

Proposition 2 gives the upper spectral bound needed below; a fixed diagonal shift will provide the lower bound in Assumption 4.

The original manuscript already intended the pair selection as a cautious update in the spirit of curvature-based cautious updates [24, 27, 28]. To make the upper test insensitive to the local gradient scale, define

$$\omega_{k_i}^{\mathrm{pair}} := \max\{\omega_{\min}, \|g_{k_i}\|, \|g_{k_i+1}\|\}, \qquad \widehat{y}_{k_i} := \frac{\overline{y}_{k_i}}{\omega_{k_i}^{\mathrm{pair}}}.$$

After damping and normalization, a pair $(s, \widehat{y})$ is retained only if

$$\widehat{y}^\top s \geq \max\left\{ \frac{\|\widehat{y}\|^2}{10^8}, 10^{-16} \right\}. \tag{33}$$

Thus every stored normalized pair satisfies eq. (31) with the iteration-independent constant $\Lambda = 10^8$. The absolute floor rejects numerically degenerate pairs but is not needed for the upper bound. Using at most $p = 10$ pairs, define $\widehat{B}_k = \mathrm{BFGS}(\{(s_{k_i}, \widehat{y}_{k_i})\})$ when the memory is nonempty and $\widehat{B}_k = I$ otherwise. We then set

$$B_k = \omega_{\min} I + \omega_k \widehat{B}_k. \tag{34}$$

By Proposition 2, $0 \prec \widehat{B}_k \preceq (1+p)\Lambda I$ for nonempty memory, while the same upper bound holds for empty memory because $(1+p)\Lambda \geq 1$. Since $\omega_{\min} \leq \omega_k$, it follows that

$$\omega_{\min} I \preceq B_k \preceq (1 + (1+p)\Lambda)\omega_k I.$$

Thus Assumption 4 holds with the iteration-independent constants $m = \omega_{\min}$ and $M = 1 + (1+p)\Lambda$. For a candidate pair with $y_{k_i}^\top s_{k_i} < 0$, the implementation applies a scalar

Powell-type damping before the cautious test. Specifically, it sets $b_i = \|y_{k_i}\|^2 / |y_{k_i}^\top s_{k_i}|$ and

$$\bar{y}_{k_i} = \theta_i^{\mathrm{damp}} y_{k_i} + (1-\theta_i^{\mathrm{damp}}) b_i s_{k_i}, \qquad \theta_i^{\mathrm{damp}} = \frac{0.8 b_i \|s_{k_i}\|^2}{b_i \|s_{k_i}\|^2 - y_{k_i}^\top s_{k_i}} \in [0,1).$$

If $y_{k_i}^\top s_{k_i} \geq 0$, it sets $\bar{y}_{k_i} = y_{k_i}$. Candidates for which this scalar damping is numerically unsafe are discarded before normalization and eq. (33). This is the damping rule used in the released code; the normalized upper test and the fixed diagonal shift, rather than damping itself, guarantee the two bounds in Assumption 4. No convexity or cocoercivity assumption on ∇f is used.

Remarks for Practical Implementation: Having established that the proposed construction of B_k satisfies Assumption 4, we now present several remarks on its practical implementation. We begin by describing how to compute the search direction

$$d_k = -(B_k + \mu_k I)^{-1} g_k$$

with B_k in eq. (34). Regularized limited-memory schemes may be employed to account for the diagonal shift [23]. Define $\widehat{\mu}_k := (\mu_k + \omega_{\min})/\omega_k$. We use the simple approximation mentioned in the same reference,

$$\widehat{B}_k + \widehat{\mu}_k I \approx \mathrm{BFGS}(\{s_{k_i}, \widehat{y}_{k_i} + \widehat{\mu}_k s_{k_i}\}_{i=0}^{p-1}). \tag{35}$$

The direction is then computed as $d_k \approx -\omega_k^{-1} \mathrm{BFGS}(\{s_{k_i}, \widehat{y}_{k_i} + \widehat{\mu}_k s_{k_i}\})^{-1} g_k$ using the standard two-loop recursion [26, 32]. Although this approximation is inexact, it has been reported to yield nearly identical practical performance while remaining comparatively simple to implement [23]. We adopt this approximation in our numerical experiments.

Separately, there exist function-based modified secant conditions [1, 20, 46, 49] that adjust the secant equation using available function values when those quantities are reliable. Such adjustments aim to improve practical convergence behavior. We partially adopt these function-based modifications following prior work [47, 50]. The suffix `-MS` in section 6 indicates the use of this technique. Because these modifications are well established, we omit low-level implementation details here. Further specifics are available in the cited references and in our open-source implementation.

5.2 Step Size α_k

We next discuss the adjustment of the step size α_k in line 4 of Algorithm 2. As indicated in the pseudo-code, we choose α_k using interpolation of the objective function. Moré–Thuente line searches [29] are widely used in quasi-Newton methods [42], and they typically employ cubic or quadratic interpolation to select trial step sizes. In our implementation, we use an essentially identical procedure, except that the trial step size is obtained by clipping it to the interval $[\beta_{\min}\alpha_k, \beta_{\max}\alpha_k]$. For all numerical experiments, we set the Armijo parameter to $c = 10^{-4}$ and choose the interpolation bounds as $\beta_{\min} = 1/16$ and $\beta_{\max} = 15/16$.

Furthermore, when $\mu_k > 0$ and $\alpha_k = 1$, we introduce a one-time adjustment of the trial step size using gradient information. Let d_k be the search direction and g_k and g_{try} the gradients at the current and trial points. If the directional derivative along d_k changes sign, $d_k^\top g_k < 0$ and $d_k^\top g_{\text{try}} > 0$, and the trial gradient is sufficiently aligned with d_k, namely $d_k^\top g_{\text{try}} > 0.5\|d_k\| \|g_{\text{try}}\|$, we rescale α_k once by a secant-like factor,

$$\alpha_k \leftarrow \operatorname{clip}\left(\alpha_k \frac{-d_k^\top g_k}{d_k^\top g_{\text{try}} - d_k^\top g_k}, \beta_{\min}\alpha_k, \beta_{\max}\alpha_k\right),$$

where $\operatorname{clip}(x,a,b) := \min(\max(x,a),b)$ is a clipping function. When objective values are unreliable and backtracking and interpolation do not occur, this correction effectively refines the step size. Since this adjustment is at most once per iteration, the theoretical results in section 4 remain valid.

5.3 Regularization Parameter μ_k

We finally discuss the adjustment of the regularization parameter μ_k in line 8 of Algorithm 1. In our implementation, the OFFO accumulator contains exactly those gradient norms generated during regularized iterations:

$$G_k^2 = \varsigma + \sum_{j\in K^+,\, j\le k} \|g_j\|^2, \qquad \mu_k = \operatorname{clip}\left(\frac{1}{10}\|g_k\|, \frac{1}{100}G_k, G_k\right).$$

Hence $\mu_k = \theta_k G_k$ with $\theta_k \in [10^{-2}, 1]$, exactly as required by eq. (7). We set $\varsigma = 10^{-20}$, so that its square root, the initial scale G_{-1}, equals 10^{-10}. The primary implementation sets the permitted number of restarts to zero and therefore coincides exactly with the analyzed accumulator. The restart was originally intended only as a practical heuristic rather than as an essential part of the proposed algorithm. For the optional heuristic, a fixed threshold $\tau > 0$ resets only the accumulator in eq. (7) whenever

$$\min_{j\in K^0,\, j<k} \left(\overline{f}(x_j) - \Delta_j\right) - \overline{f}(x_k) \ge \tau. \tag{36}$$

The sets K^0 and K^+ and the reference values in eq. (36) are not reset. Because this optional rule is heuristic, all primary experiments use the analyzed no-restart method; the variant with at most ten restarts is reserved for the separate appendix ablation in appendix C.

6 Numerical Experiments

We conducted numerical experiments to evaluate the practical performance of our proposed method and to compare it with existing optimization algorithms.

6.1 Methods

We compared the following methods in our experiments.

- (`Ours`): Our proposed regularized quasi-Newton method
  - We used Algorithms 1 and 2 with the details in section 5.
  - (`Ours-MS`) means the variant incorporating the modified secant condition described in section 5.1.
  - (`NTRQN-OFFO`) is the branch ablation that always uses the OFFO-based regularization branch.
- (`Line`): Line search L-BFGS method
  - Our Python implementation ported from LBFGSpp [33].
  - The step size was determined using the Moré–Thuente line search, closely resembling SciPy's implementation.
  - (`Line-MS`) means the same variant as `Ours-MS`.
- (`Reg`): Regularized L-BFGS method [23]
  - A quadratic regularization-based quasi-Newton method that does not rely on line search and is conceptually closer to trust-region methods.
  - We wrapped the function `regLBFGS` with `solveNonmonotone` from [23].
  - (`Reg-Sec`) denotes the variant using the approximate computation in eq. (35). It wraps the `regLBFGSsec` function from [23].
- (`SciPy`): SciPy's L-BFGS-B method [42]
  - An established L-BFGS method implemented in C and called from Python.
  - We used the default parameters, with the exception that `ftol` was set to 0 to avoid premature termination.
- (`NTQN`): Noise-tolerant quasi-Newton method [38, 45]
  - A method designed to accommodate inexact gradient information, as mentioned in section 2.3.
  - We report both the original wrapper setting and the authors' recommended termination mode (`terminate=3`).
  - For the latter, the external solver controls termination; the common gradient-tolerance criterion is used only when constructing performance profiles and is not injected into the solver.
  - The recommended mode may appear worse under a common-tolerance performance profile because it can terminate appropriately at its estimated noise neighborhood before reaching the profile's scoring tolerance; this reflects a difference in stopping objectives rather than necessarily worse iterates.
  - We therefore report both solver-declared termination and attainment of the common scoring tolerance and interpret any early truncation separately from oracle efficiency.
- (`OFFO`): A standalone first-order objective-function-free method using the cumulative gradient-norm scale in section 2.2.

In the following, each method is referred to by the name given in parentheses. Except for the first-order `OFFO` baseline, all these methods are L-BFGS-based, and we set the number of stored curvature pairs to 10, the default value in SciPy's L-BFGS-B implementation. All experiments were conducted on a Microsoft Windows 11 Home system (version 10.0, build 26100) equipped with a 13th Gen Intel® Core™ i7-1360P CPU, featuring 12 physical cores and 16 logical processors. All computations were performed using the CPU only.

6.2 Experimental Results

6.2.1 Test Problems and Settings

We evaluated all algorithms on unconstrained problems from the CUTEst benchmark collection [14], implemented in Python using the PyCUTEst interface [12]. All problems were initialized with the default starting points provided by the library. Following previous work [23], we excluded problems whose initial points were already stationary or whose function values or gradients contained `NaN` or `INF`. The 64- and 32-bit collections contain 220 problems with dimensions from 2 to 123,200, and the 16-bit collection contains 152 problems over the same dimension range. The supplied error rates are 2.22×10^{-9}, 1.19×10^{-3}, and 9.77×10^{-2} for the 64-, 32-, and 16-bit experiments, respectively. The 64-bit value equals SciPy's default heuristic $10^7 \varepsilon_{\text{mach}}$; at lower precisions we reduce the multiplier to 10^4 and 10^2, respectively, because retaining 10^7 would violate the required range $\varepsilon_{\text{f}} < 1$ and yield an uninformative error allowance. These multipliers are practical safety margins rather than estimates supplied by the theory, which motivates the misspecification study below. In addition to these precision experiments, we distinguish two additive-noise experiments on the 64-bit collection:

1. a theory-aligned setting with $\overline{f}(x) = f(x) + \xi_f$, $\xi_f \sim \text{unif}(-10^{-3}, 10^{-3})$, and exact gradients; and
2. an adverse stress test with the same function perturbation and independent componentwise gradient perturbations $\xi_g^{(i)} \sim \text{unif}(-10^{-3}, 10^{-3})$.

We supply $\varepsilon_{\text{f}} = 10^{-2}$ in both settings and additionally test 10^{-4} and 10^{-1} to assess under- and overestimation of this error parameter. Underestimating ε_{f} makes Δ_k too small to absorb the actual function error, makes the K^0 test easier to satisfy, and can therefore cause the method to trust unreliable comparisons and take more unregularized steps outside the theorem's error model. Overestimating ε_{f} enlarges Δ_k, relaxes the Armijo acceptance test, and simultaneously makes entry into K^0 more stringent through $\overline{f}(x_j) - \Delta_j$, so the expected effect is a safer but potentially more conservative use of K^+. The sensitivity experiment tests these two qualitatively different failure modes rather than treating misspecification as a purely numerical parameter sweep. Each noisy experiment uses the five seeds $0, 1, 2, 3, 4$, records the seed, and supplies the same seed to all solvers for a given problem and scenario. The joint-noise setting deliberately violates eq. (3); it is included to test performance under an unfavorable perturbation, not to validate the exact-gradient theorem. This setting is particularly adverse for our method because its theory and regularization logic rely on accurate gradient information, whereas NTQN was designed specifically to accommodate bounded gradient errors.

Each setting uses a different error rate ε_{f} in the error model (eq. (2)), as summarized in Table 2. The table relates the machine epsilon (maximum relative rounding

explanation	min abs value	relative error	ε_{f}
64-bit double precision	2.23×10^{-308}	$2^{-52} \approx 2.22 \times 10^{-16}$	2.22×10^{-9}
32-bit single precision	1.18×10^{-38}	$2^{-23} \approx 1.19 \times 10^{-7}$	1.19×10^{-3}
16-bit half precision	6.10×10^{-5}	$2^{-10} \approx 9.77 \times 10^{-4}$	9.77×10^{-2}

Table 2: The “min abs value” is the smallest positive normalized number, the “relative error” is the maximum relative rounding error, and ε_{f} is the parameter in (2) used in our experiments.

error) for each floating-point precision to the ε_{f} value used in our experiments. The relatively large ε_{f} values are chosen to account for the cumulative errors that may arise during optimization. We also set the maximum number of iterations to $k_{\max} = 15{,}000$ and timed out runs exceeding 10 minutes per problem.

6.2.2 Performance Profiles

We used performance profiles [10] to compare the algorithms. Let P denote the set of test problems and S the set of solvers. For each problem $p \in P$ and solver $s \in S$, let $n_{p,s} > 0$ denote the number of oracle calls (i.e., function and gradient evaluations) required to reach a specified gradient norm tolerance $\varepsilon_{\mathrm{gtol}}$. In this experiment, we use the number of oracle calls rather than clock time to avoid the influence of algorithmic overhead and to focus on the intrinsic optimization efficiency. When a solver fails to converge within the gradient tolerance or exceeds a maximum number of oracle calls, we set $n_{p,s} = +\infty$. Then, the performance ratio $r_{p,s}$ is defined as

$$r_{p,s} = \frac{n_{p,s}}{\min_{s' \in S} n_{p,s'}}.$$

For a given threshold $\tau \geq 1$ and solver s, the performance profile plots the proportion of problems for which $r_{p,s} \leq \tau$. Thus, the y-coordinate for solver s at x-coordinate τ is given by

$$\rho_s(\tau) = \frac{1}{|P|} \left| \left\{ p \in P \;\middle|\; r_{p,s} \leq \tau \right\} \right|.$$

A curve that lies above others indicates better performance. Following standard implementations [42], we used the infinity norm for the gradient, though other norms may equally be employed.

6.2.3 Results with Artificial Noise

We first separate the theory-aligned function-noise experiment from the joint-noise stress test described in section 6.2.1. The former tests the assumptions of section 4; the latter asks whether the implementation remains useful when both available quantities are perturbed. Therefore, success in the joint-noise experiment is evidence that the method can remain effective even in a setting that is theoretically unfavorable to it, but it is not presented as validation of the exact-gradient complexity result. The experiment notebook compares the hybrid method with both the standalone first-order

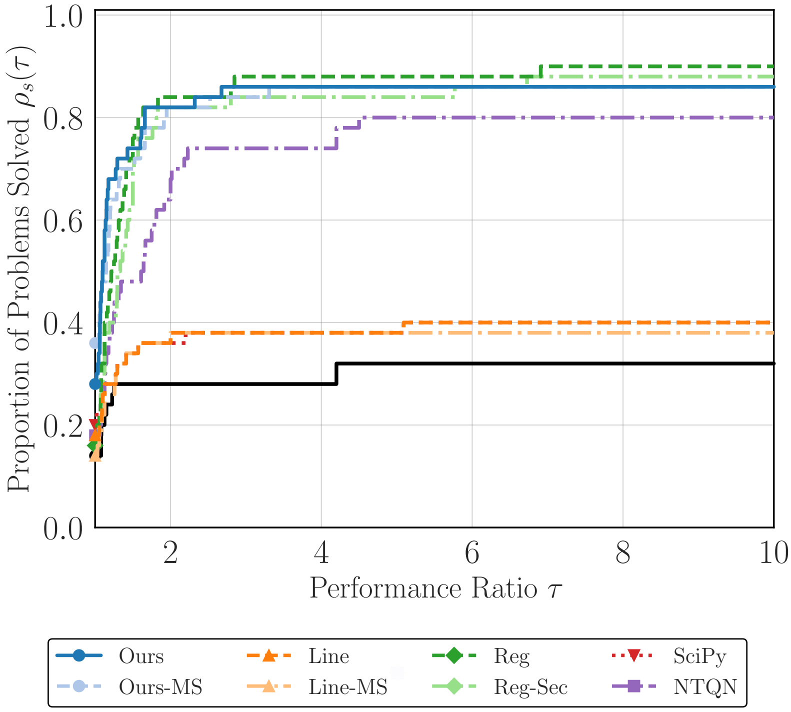
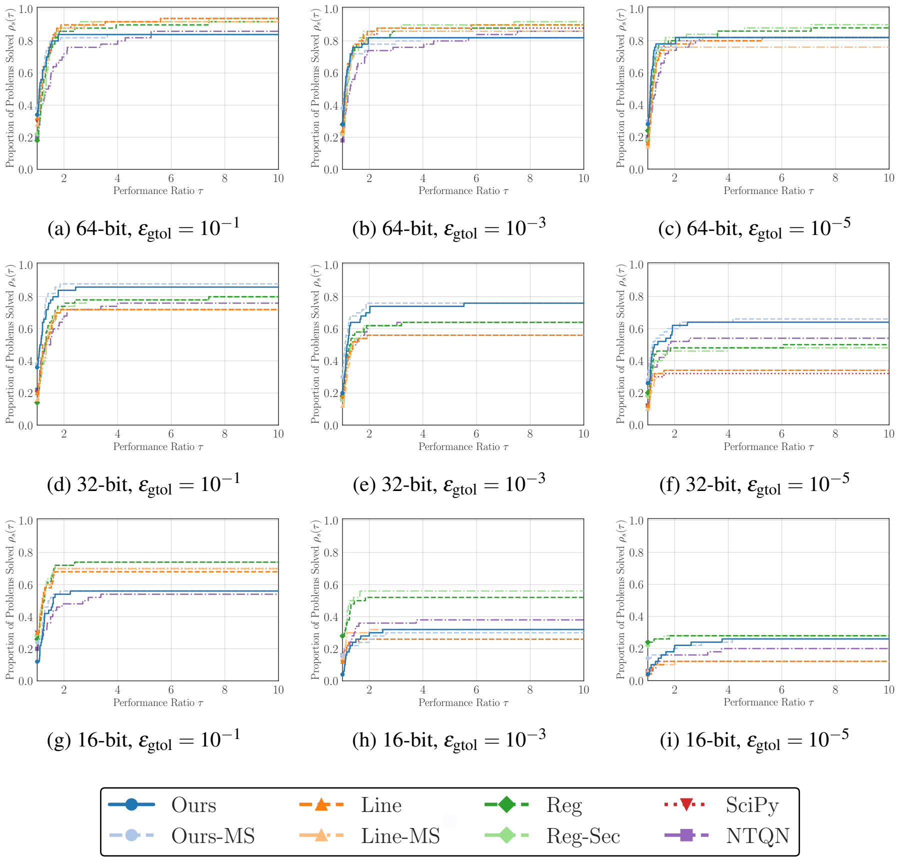


Fig. 1: Performance profile for the deliberately adverse joint function-and-gradient noise setting (componentwise level 10^{-3}) and $\varepsilon_{\mathrm{gtol}} = 10^{-2}$; this experiment lies outside the exact-gradient theory.

`OFFO` baseline and the always-OFFO branch ablation `NTRQN-OFFO`, and varies the supplied ε_{f} over two orders of magnitude around its nominal value. All main-text profiles use the analyzed no-restart method; the earlier accumulator-restart heuristic is evaluated separately in the appendix.

6.2.4 Results at Different Precision Levels

We next present performance profiles for each floating-point precision (64-, 32-, and 16-bit). We evaluated each method at three gradient norm tolerance levels, i.e., $\varepsilon_{\mathrm{gtol}} \in \{10^{-1}, 10^{-3}, 10^{-5}\}$. These tolerance levels represent different optimization regimes, from moderate accuracy to high precision requirements. Figure 2 shows the comprehensive performance profiles across all three precision levels. The profiles show that `Ours` and `Ours-MS` solve a large fraction of the retained problems across the three precision levels. Their relative ranking depends on the tolerance and precision, so we avoid claiming uniform superiority and report both the fraction solved and the performance ratios in Figure 2.



(a) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (b) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (c) 64-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

(d) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (e) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (f) 32-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

(g) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-1}$ (h) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-3}$ (i) 16-bit, $\varepsilon_{\text{gtol}} = 10^{-5}$

Fig. 2: Performance profiles across floating-point precisions (64-, 32-, and 16-bit) and tolerance levels.

6.3 Computational Time per Iteration

In this subsection, we present an experiment comparing the per-iteration computational cost of each method. Here, we measure total execution time rather than the number of oracle calls. We conducted experiments on the ill-conditioned quadratic problem

$$\underset{x\in\mathbb{R}^n}{\text{minimize}} \quad \frac{1}{2}\sum_{i=1}^{n} i x_i^2$$

with $n = 10{,}000$. The initial point x_0 was chosen as the all-ones vector. We used memory size $m = 10$ and maximum iterations $k_{\max} = 100$, and we did not set any stopping criteria other than reaching $k_{\max}$. The simplicity of the optimization problem and the small value of $k_{\max}$ are well suited to the purpose of this experiment, as they ensure that the total runtime is dominated by algorithmic overhead rather than

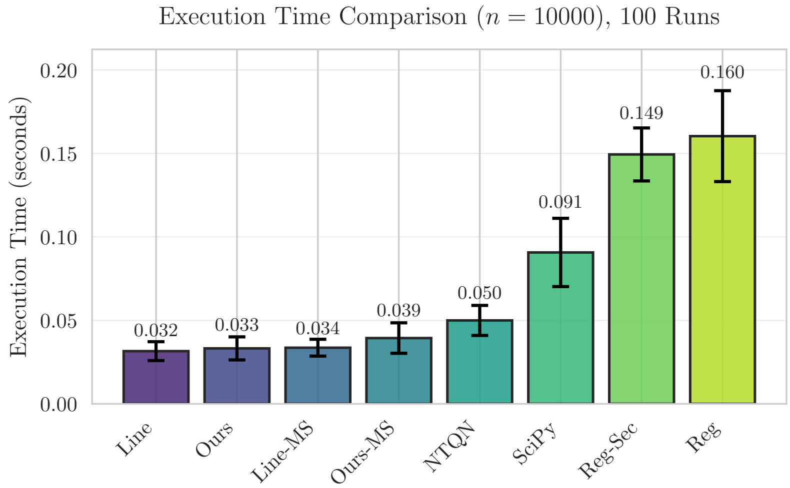


Fig. 3: Mean execution times for the experiment in section 6.3. This plot roughly indicates the extra computational overhead of each method. Error bars represent one standard deviation over 100 runs.

	`Line`	`Ours`	`Line-MS`	`Ours-MS`	`NTQN`	`SciPy`	`Reg-Sec`	`Reg`
# Calls	212	202	212	202	219	212	206	206
$\overline{f}(x_{k_{\max}})$	1.24	1.34	1.24	1.34	1.43	1.24	1.48	1.47

Table 3: The number of oracle calls and final observed objective values for the experiment in section 6.3.

function or gradient computation. Timing data were collected after three warm-up runs, followed by 100 recorded trials.

In Figure 3, we report the execution time statistics, which show that the per-iteration times of `Ours` and `Ours-MS` are of the same order as those of the other Python implementations in this isolated test. This timing experiment measures algorithmic overhead under deliberately inexpensive objective and gradient evaluations and does not by itself establish end-to-end robustness or superiority in oracle cost. Because wall-clock time depends strongly on implementation details and the computing environment, we restrict its interpretation to this isolated experiment and do not draw broader conclusions from the timing results. For a fairer comparison of algorithmic efficiency, we place greater emphasis on oracle-call counts and success profiles.

In Table 3, we report the total number of oracle calls (calls of $\overline{f}$ and g) and final objective values $\overline{f}(x_{k_{\max}})$. The purpose of this table is to check whether the methods achieved similar convergence behavior within the limited number of iterations. Comparable values would indicate that differences in convergence behavior do not dominate the execution-time comparison.

7 Conclusion

We proposed a hybrid regularized quasi-Newton method for smooth nonconvex optimization with bounded errors in objective-function values. Under exact gradients, a fixed positive lower spectral bound, and a gradient-scaled upper spectral bound on the quasi-Newton matrices, the method retains the order-optimal $\mathscr{O}(1/\varepsilon^2)$ first-order iteration complexity of the noise-free L-smooth nonconvex setting while selectively using noisy function values. The implementation enforces the normalized upper-curvature condition, supplies the lower bound through a fixed diagonal shift, and uses the same non-restarted OFFO accumulator as the analysis. The experiments distinguish function-value noise covered by the theorem from a joint-noise stress test outside its scope and assess sensitivity to the supplied error bound. An analysis with inexact gradients, which can at best guarantee convergence to a noise-dependent neighborhood without additional assumptions, remains future work.

A Proof of Lemma 6

The proof of Lemma 6 is similar to existing results [44, Lemma 3.2], [17, Lemma 3.1].

Proof Recall that eq. (7) defines μ_k as $\theta_k \sqrt{\varsigma + \sum_{j \in K^+, j \leq k} \|g_j\|^2}$, and line 8 of Algorithm 1 asserts $\theta_k \in [\theta_{\min}, \theta_{\max}]$. Let $k_i \in K^+$ be the i-th element of K^+. For any $0 < x < y$, $x/y \leq -\ln(1 - x/y) = \ln y - \ln(y - x)$ holds. Thus, we have

$$\begin{aligned}
\frac{\|g_{k_i}\|^2}{\mu_{k_i}^2} &\leq \frac{1}{\theta_{\min}^2} \frac{\|g_{k_i}\|^2}{\varsigma + \sum_{\ell=0}^{i} \|g_{k_\ell}\|^2} && \text{(definition)} \\
&\leq \frac{1}{\theta_{\min}^2} \left(\ln \left(\varsigma + \sum_{\ell=0}^{i} \|g_{k_\ell}\|^2 \right) - \ln \left(\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2 \right) \right). && (x/y \leq \ln y - \ln(y-x))
\end{aligned}$$

Similarly, $x/\sqrt{y} \geq x/(\sqrt{y} + \sqrt{y-x}) = \sqrt{y} - \sqrt{y-x}$ holds. Thus, we have

$$\begin{aligned}
\frac{\|g_{k_i}\|^2}{\mu_{k_i}} &\geq \frac{1}{\theta_{\max}} \frac{\|g_{k_i}\|^2}{\sqrt{\varsigma + \sum_{\ell=0}^{i} \|g_{k_\ell}\|^2}} && \text{(definition)} \\
&\geq \frac{1}{\theta_{\max}} \left(\sqrt{\varsigma + \sum_{\ell=0}^{i} \|g_{k_\ell}\|^2} - \sqrt{\varsigma + \sum_{\ell=0}^{i-1} \|g_{k_\ell}\|^2} \right). && (x/\sqrt{y} \geq \sqrt{y} - \sqrt{y-x})
\end{aligned}$$

Summing these inequalities over $0 \leq i < |K^+|$ yields the claim. □

B Proof of Proposition 2

The proof of Proposition 2 uses the standard BFGS update.

Proof From eq. (31), the initial scalar satisfies $0 < \gamma = \|y_{k_0}\|^2/(y_{k_0}^\top s_{k_0}) \le \Lambda$. Let B and B_+ denote the positive definite Hessian approximations before and after one update. The BFGS update rule is

$$B_+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{yy^\top}{y^\top s}.$$

Positive definiteness follows from $B \succ 0$, $s \neq 0$, and $y^\top s > 0$. Moreover, eq. (31) and the fact that $Bss^\top B/(s^\top Bs)$ is positive semidefinite give

$$B_+ \preceq B + \frac{yy^\top}{y^\top s} \preceq B + \Lambda I.$$

Applying this inequality recursively over p updates from $B^{(0)} = \gamma I \preceq \Lambda I$ proves eq. (32). $\square$

The normalized upper-curvature inequality is enforced directly by eq. (33), while the fixed shift in eq. (34) supplies the lower spectral bound. No convexity or cocoercivity assumption is used. This point is important because the convergence theorem permits nonconvex objectives.

C Accumulator-Restart Ablation

This appendix isolates the earlier accumulator-restart heuristic from the primary no-restart experiments that match Algorithm 1 and the analysis in section 4. The restart variant uses the condition in eq. (36), permits at most ten restarts, and otherwise uses the same problems, seeds, stopping rules, and parameter values as the corresponding no-restart method. For each setting, the comparison records oracle calls, failures, the distribution of the number of restarts, and whether the cap is reached.

References

1. Babaie-Kafaki, S., Ghanbari, R., Mahdavi-Amiri, N.: Two new conjugate gradient methods based on modified secant equations. Journal of Computational and Applied Mathematics **234**(5), 1374–1386 (2010). DOI 10.1016/j.cam.2010.01.052
2. Bellavia, S., Gurioli, G., Morini, B., Toint, P.L.: The Impact of Noise on Evaluation Complexity: The Deterministic Trust-Region Case. Journal of Optimization Theory and Applications **196**(2), 700–729 (2023). DOI 10.1007/s10957-022-02153-5
3. Berahas, A.S., Cao, L., Choromanski, K., Scheinberg, K.: A Theoretical and Empirical Comparison of Gradient Approximations in Derivative-Free Optimization. Foundations of Computational Mathematics **22**(2), 507–560 (2022). DOI 10.1007/s10208-021-09513-z

4. Berahas, A.S., Curtis, F.E., Zhou, B.: Limited-memory BFGS with displacement aggregation. Mathematical Programming **194**(1), 121–157 (2022). DOI 10.1007/s10107-021-01621-6
5. Caflisch, R.E.: Monte Carlo and quasi-Monte Carlo methods. Acta Numerica **7**, 1–49 (1998). DOI 10.1017/S0962492900002804
6. Carter, R.G.: Numerical Experience with a Class of Algorithms for Nonlinear Optimization Using Inexact Function and Gradient Information. SIAM Journal on Scientific Computing **14**(2), 368–388 (1993). DOI 10.1137/0914023
7. Clancy, R.J., Menickelly, M., Hückelheim, J., Hovland, P., Nalluri, P., Gjini, R.: TROPHY: Trust Region Optimization Using a Precision Hierarchy. In: Computational Science – ICCS 2022, pp. 445–459. Springer, Cham (2022). DOI 10.1007/978-3-031-08751-6_32
8. Corless, R.M., Gonnet, G.H., Hare, D.E.G., Jeffrey, D.J., Knuth, D.E.: On the Lambert W function. Advances in Computational Mathematics **5**(1), 329–359 (1996). DOI 10.1007/BF02124750
9. Doikov, N., Mishchenko, K., Nesterov, Y.: Super-Universal Regularized Newton Method. SIAM Journal on Optimization **34**(1), 27–56 (2024). DOI 10.1137/22M1519444
10. Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. Mathematical Programming **91**(2), 201–213 (2002). DOI 10.1007/s101070100263
11. Duchi, J., Hazan, E., Singer, Y.: Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. Journal of Machine Learning Research **12**(61), 2121–2159 (2011). URL http://jmlr.org/papers/v12/duchi11a.html
12. Fowkes, J., Roberts, L., Bűrmen, Á.: PyCUTEst: An open source Python package of optimization test problems. Journal of Open Source Software **7**(78), 4377 (2022). DOI 10.21105/joss.04377
13. Gao, W., Goldfarb, D.: Quasi-Newton Methods: Superlinear Convergence Without Line Searches for Self-Concordant Functions. arXiv preprint arXiv:1612.06965 (2018). DOI 10.48550/arXiv.1612.06965
14. Gould, N.I., Orban, D., Toint, P.L.: CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for mathematical optimization. Comput. Optim. Appl. **60**(3), 545–557 (2015). DOI 10.1007/s10589-014-9687-3
15. Gower, R.M., Goldfarb, D., Richtárik, P.: Stochastic Block BFGS: Squeezing More Curvature out of Data. arXiv preprint arXiv:1603.09649 (2016). DOI 10.48550/arXiv.1603.09649
16. Grapiglia, G.N., Stella, G.F.D.: An adaptive trust-region method without function evaluations. Computational Optimization and Applications **82**(1), 31–60 (2022). DOI 10.1007/s10589-022-00356-0
17. Gratton, S., Jerad, S., Toint, Ph.L.: Complexity of a class of first-order objective-function-free optimization algorithms. Optimization Methods and Software **41**(2), 478–508 (2026). DOI 10.1080/10556788.2023.2296431
18. Gratton, S., Toint, P.L.: A note on solving nonlinear optimization problems in variable precision. Computational Optimization and Applications **76**(3), 917–933 (2020). DOI 10.1007/s10589-020-00190-2

19. Gratton, S., Toint, P.L.: OFFO minimization algorithms for second-order optimality and their complexity. Computational Optimization and Applications **84**(2), 573–607 (2023). DOI 10.1007/s10589-022-00435-2
20. Hassan, B.A., Moghrabi, I.A.R.: A modified secant equation quasi-Newton method for unconstrained optimization. Journal of Applied Mathematics and Computing **69**(1), 451–464 (2023). DOI 10.1007/s12190-022-01750-x
21. Higham, N.J.: Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics (2002). DOI 10.1137/1.9780898718027
22. Izycheva, A., Darulova, E.: On sound relative error bounds for floating-point arithmetic. In: Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design. FMCAD Inc, Austin, Texas (2017)
23. Kanzow, C., Steck, D.: Regularization of limited memory quasi-Newton methods for large-scale nonconvex minimization. Mathematical Programming Computation **15**(3), 417–444 (2023). DOI 10.1007/s12532-023-00238-4
24. Li, D.H., Fukushima, M.: On the Global Convergence of the BFGS Method for Nonconvex Unconstrained Optimization Problems. SIAM Journal on Optimization **11**(4), 1054–1064 (2001). DOI 10.1137/S1052623499354242
25. Li, Y.J., Li, D.H.: Truncated regularized Newton method for convex minimizations. Computational Optimization and Applications **43**(1), 119–131 (2009). DOI 10.1007/s10589-007-9128-7
26. Liu, D.C., Nocedal, J.: On the limited memory BFGS method for large scale optimization. Mathematical Programming **45**(1), 503–528 (1989). DOI 10.1007/BF01589116
27. Mannel, F.: A Globalization of L-BFGS and the Barzilai–Borwein Method for Nonconvex Unconstrained Optimization. Journal of Optimization Theory and Applications **204**(3), 1–34 (2025). DOI 10.1007/s10957-024-02565-5
28. Mannel, F., Om Aggrawal, H., Modersitzki, J.: A structured L-BFGS method and its application to inverse problems. Inverse Problems **40**(4), 045022 (2024). DOI 10.1088/1361-6420/ad2c31
29. Moré, J.J., Thuente, D.J.: Line search algorithms with guaranteed sufficient decrease. ACM Trans. Math. Softw. **20**(3), 286–307 (1994). DOI 10.1145/192115.192132
30. Moré, J.J., Wild, S.M.: Estimating Computational Noise. SIAM Journal on Scientific Computing **33**(3), 1292–1314 (2011). DOI 10.1137/100786125
31. Nesterov, Y., Polyak, B.: Cubic regularization of Newton method and its global performance. Mathematical Programming **108**(1), 177–205 (2006). DOI 10.1007/s10107-006-0706-8
32. Nocedal, J., Wright, S.J.: Numerical Optimization. Springer (1999). DOI 10.1007/978-0-387-40065-5
33. Okazaki, N.: libLBFGS: A library of limited-memory broyden-fletcher-goldfarb-shanno (l-BFGS) (2007). URL https://github.com/chokkan/liblbfgs
34. Paquette, C., Scheinberg, K.: A Stochastic Line Search Method with Expected Complexity Analysis. SIAM Journal on Optimization **30**(1), 349–376 (2020). DOI 10.1137/18M1216250
35. Ranganath, A., Singhal, M., Marcia, R.: Symmetric Rank-One Quasi-Newton Methods for Deep Learning Using Cubic Regularization. In: 2025 IEEE 10th

International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP), pp. 121–125 (2025). DOI 10.1109/CAMSAP66162.2025.11423871
36. Ruan, H., Yang, W.: Adaptive Regularized Quasi-Newton Method Using Inexact First-Order Information. Journal of Computational Mathematics **42**(6), 1656–1687 (2024). DOI 10.4208/jcm.2306-m2022-0279
37. Sheng, Z., Yuan, G., Cui, Z.: A new adaptive trust region algorithm for optimization problems. Acta Mathematica Scientia **38**(2), 479–496 (2018). DOI 10.1016/S0252-9602(18)30762-8
38. Shi, H.J.M., Xie, Y., Byrd, R., Nocedal, J.: A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization. SIAM Journal on Optimization **32**(1), 29–55 (2022). DOI 10.1137/20M1373190
39. Sun, S., Nocedal, J.: A trust region method for noisy unconstrained optimization. Mathematical Programming **202**(1), 445–472 (2023). DOI 10.1007/s10107-023-01941-9
40. Ueda, K., Yamashita, N.: Convergence Properties of the Regularized Newton Method for the Unconstrained Nonconvex Optimization. Applied Mathematics and Optimization **62**(1), 27–46 (2010). DOI 10.1007/s00245-009-9094-9
41. Ueda, K., Yamashita, N.: A regularized Newton method without line search for unconstrained optimization. Computational Optimization and Applications **59**(1), 321–351 (2014). DOI 10.1007/s10589-014-9656-x
42. Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S.J., Brett, M., Wilson, J., Millman, K.J., Mayorov, N., Nelson, A.R.J., Jones, E., Kern, R., Larson, E., Carey, C.J., Polat, I., Feng, Y., Moore, E.W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E.A., Harris, C.R., Archibald, A.M., Ribeiro, A.H., Pedregosa, F., van Mulbregt, P., SciPy 1.0 Contributors: SciPy 1.0: Fundamental algorithms for scientific computing in python. Nature Methods **17**, 261–272 (2020). DOI 10.1038/s41592-019-0686-2
43. Wang, S., Fadili, J., Ochs, P.: Convergence rates of regularized quasi-Newton methods without strong convexity. arXiv preprint arXiv:2506.00521 (2025). DOI 10.48550/arXiv.2506.00521
44. Ward, R., Wu, X., Bottou, L.: AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. Journal of Machine Learning Research **21**(219), 1–30 (2020). URL http://jmlr.org/papers/v21/18-352.html
45. Xie, Y., Byrd, R.H., Nocedal, J.: Analysis of the BFGS Method with Errors. SIAM Journal on Optimization **30**(1), 182–209 (2020). DOI 10.1137/19M1240794
46. Yabe, H., Ogasawara, H., Yoshino, M.: Local and superlinear convergence of quasi-Newton methods based on modified secant conditions. Journal of Computational and Applied Mathematics **205**(1), 617–632 (2007). DOI 10.1016/j.cam.2006.05.018
47. Yuan, G., Wei, Z.: Convergence analysis of a modified BFGS method on convex minimizations. Computational Optimization and Applications **47**(2), 237–255 (2010). DOI 10.1007/s10589-008-9219-0

48. Zhang, H., Ni, Q.: A new regularized quasi-Newton method for unconstrained optimization. Optimization Letters **12**(7), 1639–1658 (2018). DOI 10.1007/s11590-018-1236-z
49. Zhang, J., Xu, C.: Properties and numerical performance of quasi-Newton methods with modified quasi-Newton equations. Journal of Computational and Applied Mathematics **137**(2), 269–278 (2001). DOI 10.1016/S0377-0427(00)00713-5
50. Zhang, J.Z., Deng, N.Y., Chen, L.H.: New Quasi-Newton Equation and Related Methods for Unconstrained Optimization. Journal of Optimization Theory and Applications **102**(1), 147–167 (1999). DOI 10.1023/A:1021898630001

Statements and Declarations

Funding

This work was partially supported by JSPS KAKENHI (23H03351, 24K23853) and JST CREST (JPMJCR24Q2).

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Author Contributions

All authors contributed to the study conception and design. The first draft of the manuscript was written by Hiroki Hamaguchi, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Data Availability

All data analyzed during this study are publicly available. URLs are included in this article.

Code Availability

The code used in this study is publicly available at the following GitHub repository: https://github.com/HirokiHamaguchi/qnlab.